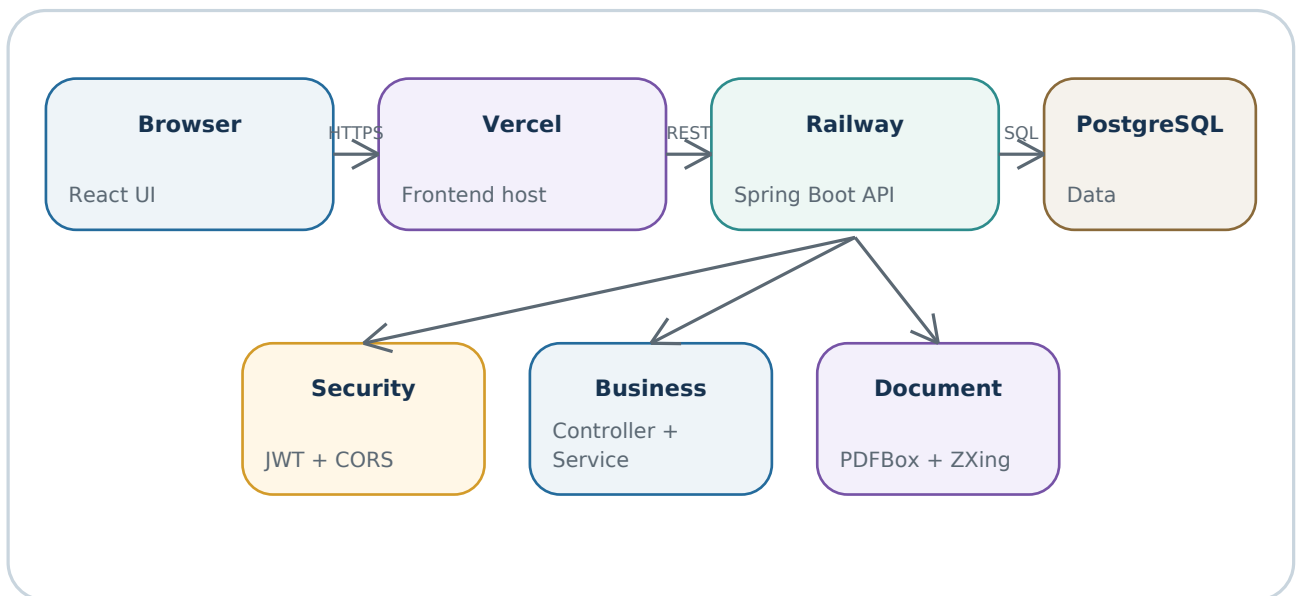


HAL Smart PDF Approval & QR Verification System

Architecture • Implementation • Security • Database • PDF/QR •
Deployment • Debugging



How to use this guide

Read the diagrams first. Then follow each arrow as a request travels through the system. For every component ask: What is it? Why is it here? What calls it? What can fail?

Contents

01. System at a glance

mental model

02. Technology stack

responsibility map

03. Frontend architecture

React + routing + Axios

04. Spring Boot architecture

security → controller → service → repository

05. Authentication & JWT

login lifecycle

06. Database design

entities + relationships

07. Document workflow

upload → review → approval

08. PDFBox + ZXing

approved PDF generation

09. Security design

JWT, BCrypt, CORS

10. Docker local architecture

three-service environment

11. Production deployment

Vercel + Railway

12. Debugging playbook

symptom → evidence → fix

13. Interview-ready explanation

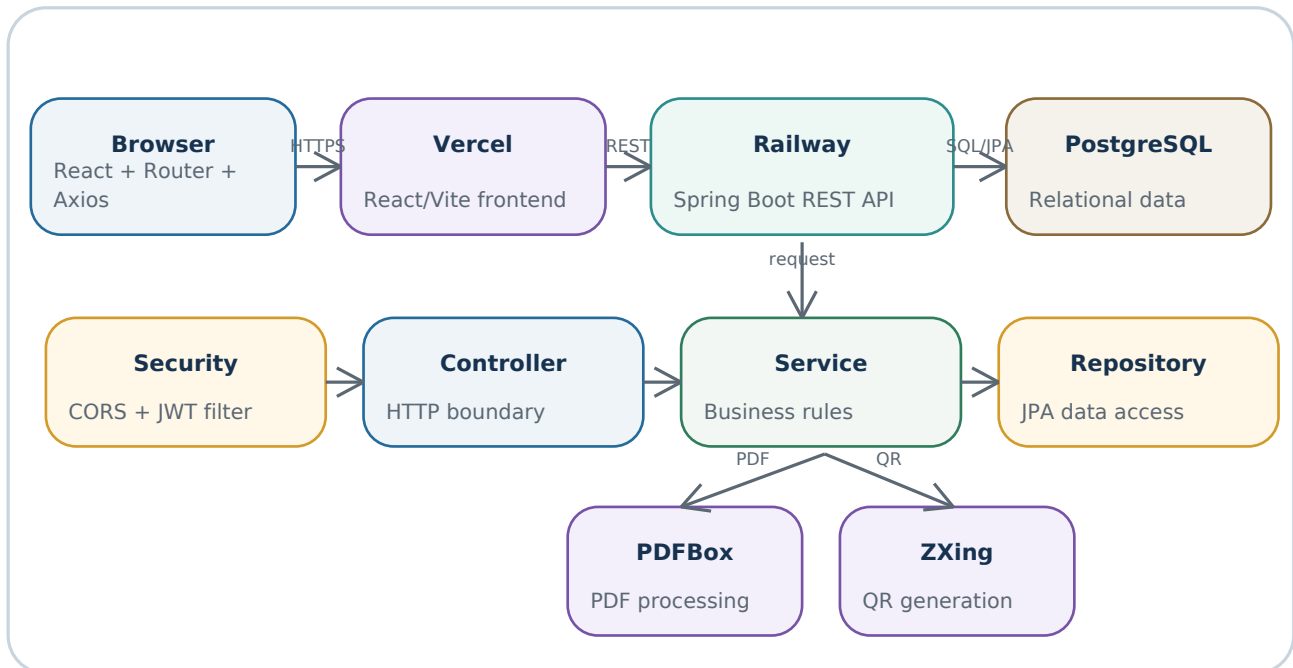
draw + explain

Core principle

The frontend improves usability, but the backend is the trust boundary. Authentication and authorization must be enforced server-side.

01 — System at a glance

The application is a full-stack document approval system. Employees authenticate and upload PDF documents. Documents enter a pending state. An administrator reviews each pending document and approves or rejects it. Approval triggers PDF/QR processing, producing an approved artifact that can be downloaded and verified.



Mental model

Think in five layers: Client → Security → Business → Persistence → Document infrastructure.
Deployment adds the hosting boundary around those layers.

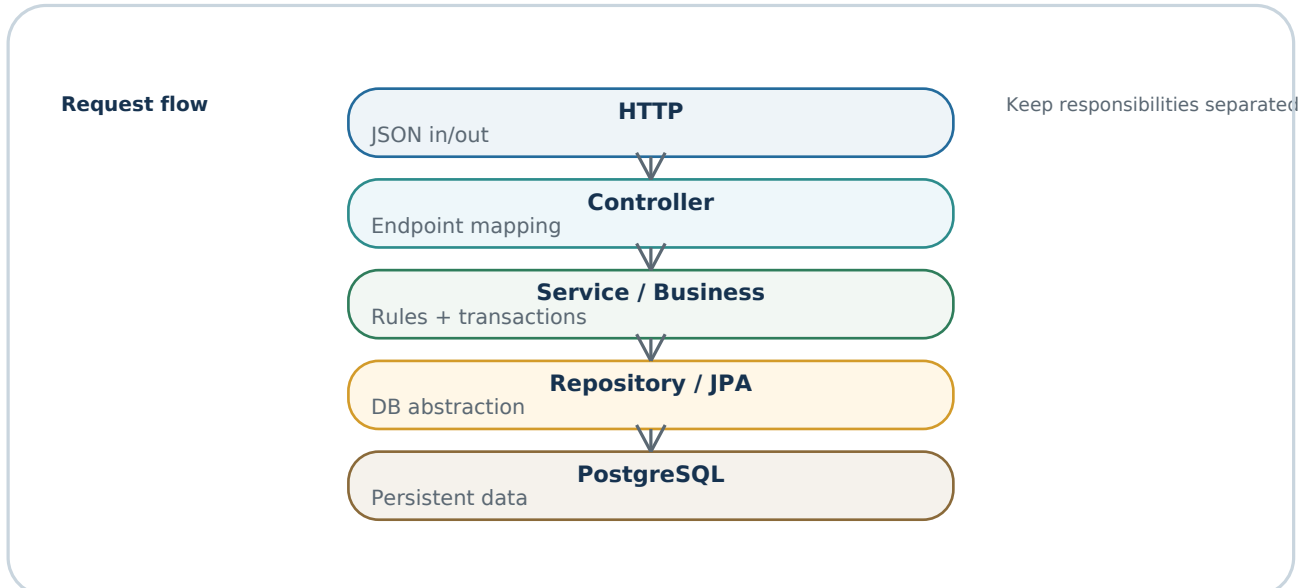
A request is not simply “frontend calls database.” The browser calls the REST API. Spring Security evaluates the request, the controller maps it, the service applies business rules, repositories access PostgreSQL, and specialized components handle PDFs and QR codes.

02 — Technology stack & responsibility

Technology	Where it fits	What to understand
Java 21	Backend runtime	JVM, OOP, exceptions, collections
Spring Boot 3.5.16	REST backend	IoC, DI, MVC, configuration
Spring Security	Security	filter chain, authentication, authorization, CORS
JWT + BCrypt	Authentication	signed token + one-way password hashing
JPA / Hibernate	Persistence	ORM mapping and SQL generation
PostgreSQL	Database	PK/FK, constraints, transactions
PDFBox 3.0.3	PDF processing	load, modify, save
ZXing 3.5.3	QR generation	encode verification payload
React + Vite	Frontend	components and production build
React Router	Navigation	client-side routes
Axios	API client	central backend base URL
Docker Compose	Local environment	multi-container orchestration
Vercel / Railway	Production	frontend / backend + database

03 — Frontend architecture

The React frontend owns presentation, navigation and API interaction. AppRoutes maps URLs to pages. ProtectedRoute checks whether a token exists before allowing navigation to protected pages. Axios centralizes the API base URL through VITE_API_URL.



Important distinction: ProtectedRoute is not a security boundary. A user can modify client-side JavaScript or call the API directly. The backend must validate JWTs and enforce authorization for every protected operation.

Route map

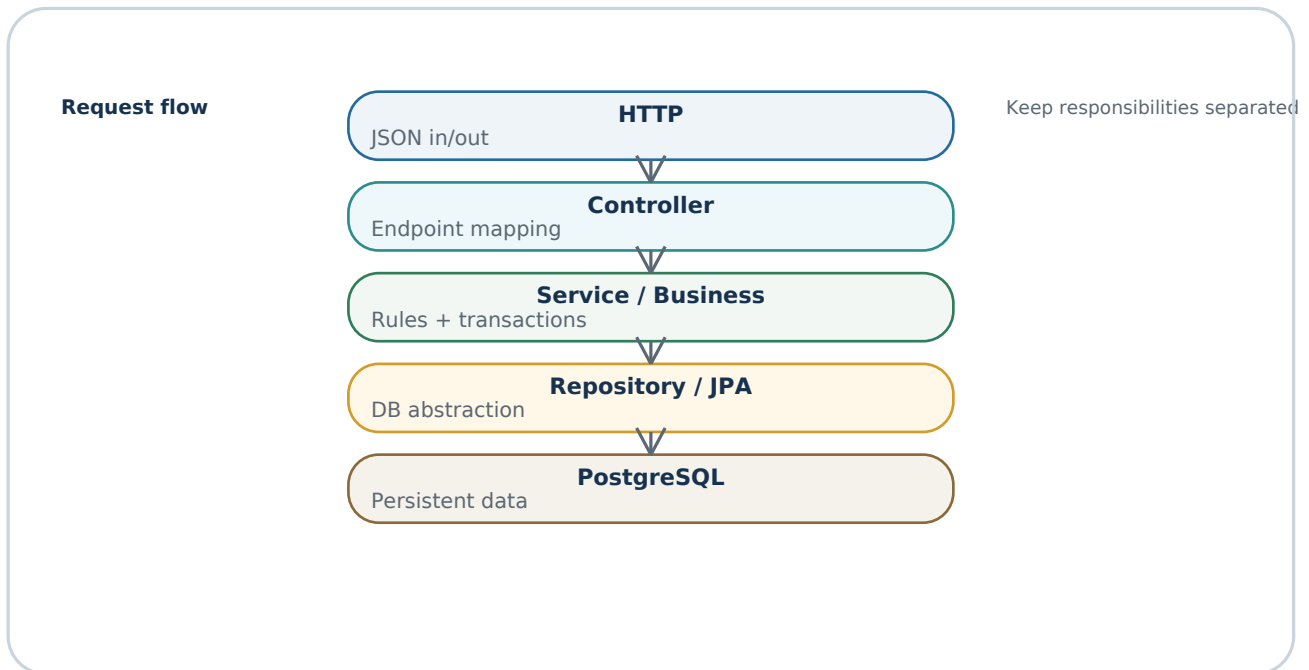
/ → public demo /login → login /dashboard → protected dashboard /upload → protected upload
/documents → protected document management

Interview line

“React controls the user experience; Spring Security controls whether the request is actually trusted.”

04 — Spring Boot architecture

The backend is intentionally layered so each class has a focused responsibility. Controllers are the HTTP boundary, services contain business rules, repositories isolate persistence, and security filters handle authentication before protected controllers run.



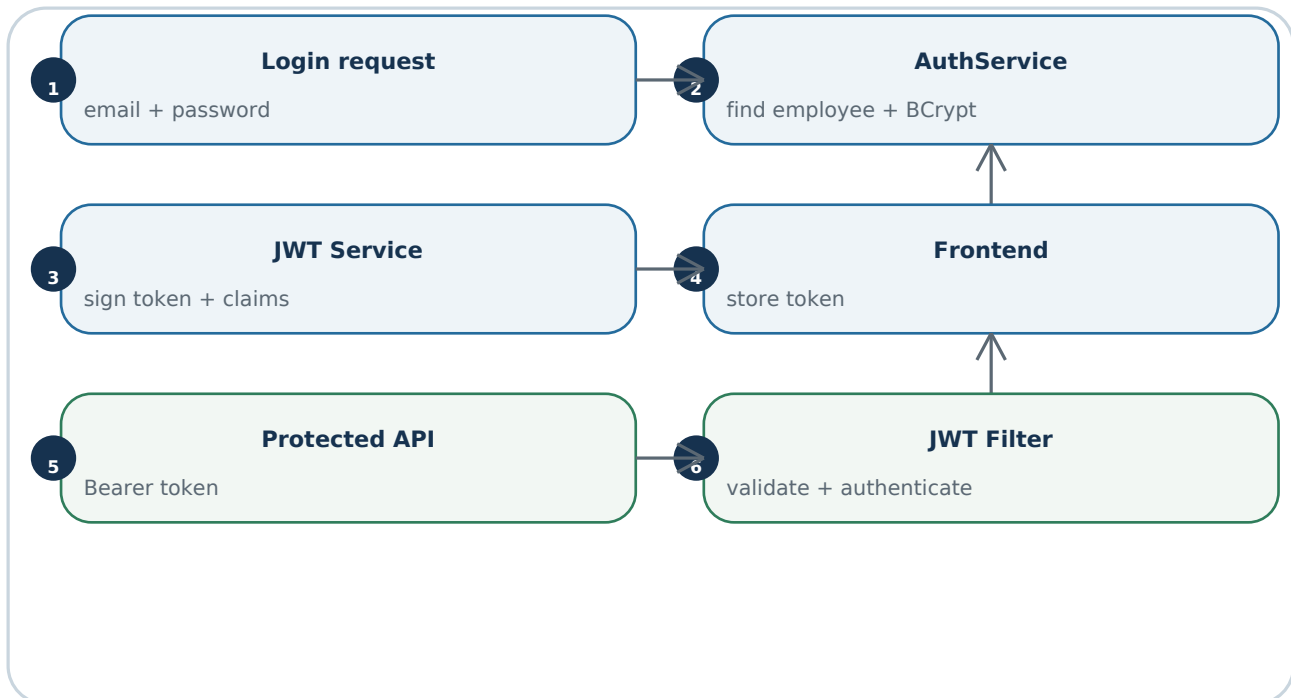
Typical request path

HTTP request → Security filter chain → JWT authentication → Controller → Service → Repository/JPA → PostgreSQL → response

- Controller: receives request data and returns an HTTP response.
- Service: decides what the application should do and coordinates multiple operations.
- Repository: provides database access through Spring Data JPA.
- Entity: represents persisted state and relationships.
- Security filter: extracts and validates the JWT before the protected endpoint executes.

05 — Authentication & JWT lifecycle

Login is a two-stage process: credentials are verified, then a signed token is issued. Later API requests carry that token so the backend can reconstruct the authenticated identity.



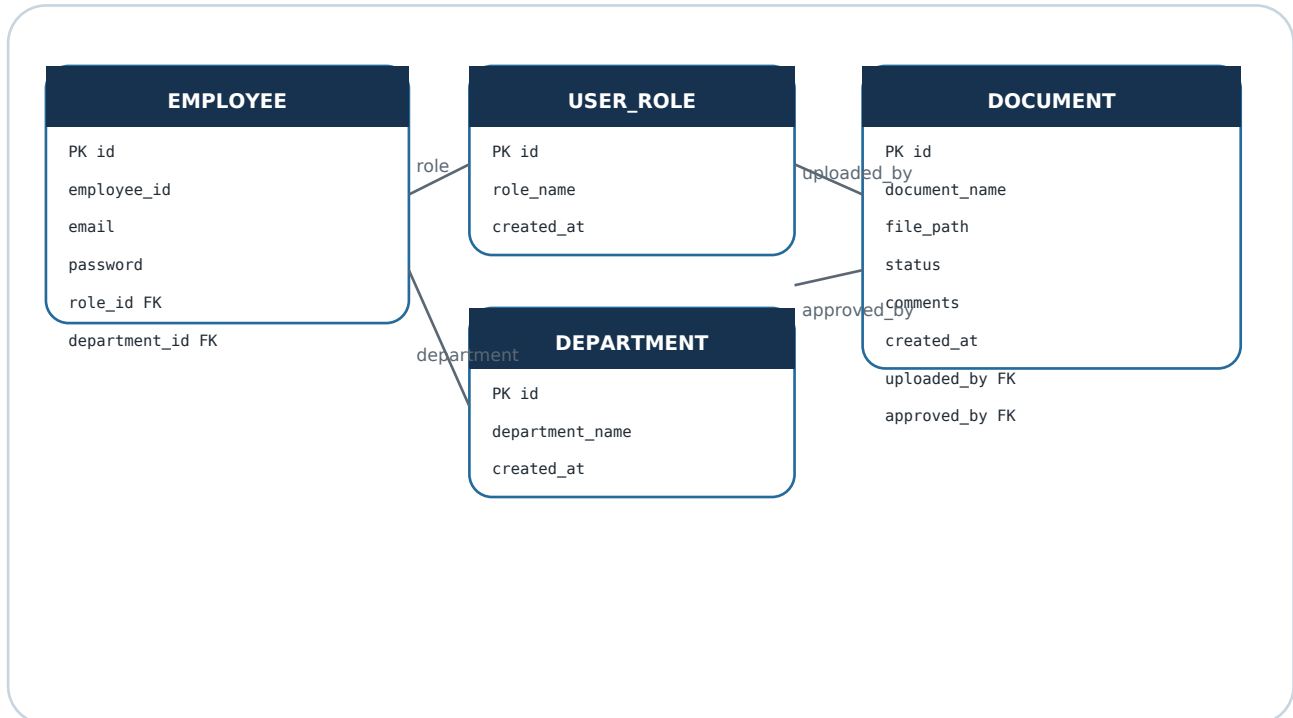
JWT payload in this project should be understood as a signed identity/authorization carrier. The token is not a database row and should not contain secrets. Its validity depends on the signing secret and expiration.

Failure modes

Empty/weak `JWT_SECRET` can prevent application startup or token generation. A missing/invalid token causes protected requests to be rejected. A valid token does not automatically mean the user is allowed to perform every operation; authorization still matters.

06 — Database design

The relational model separates people, roles, departments and documents. Employee records reference a role and department. Documents reference the employee who uploaded them and, when applicable, the employee who approved them.



Key tables

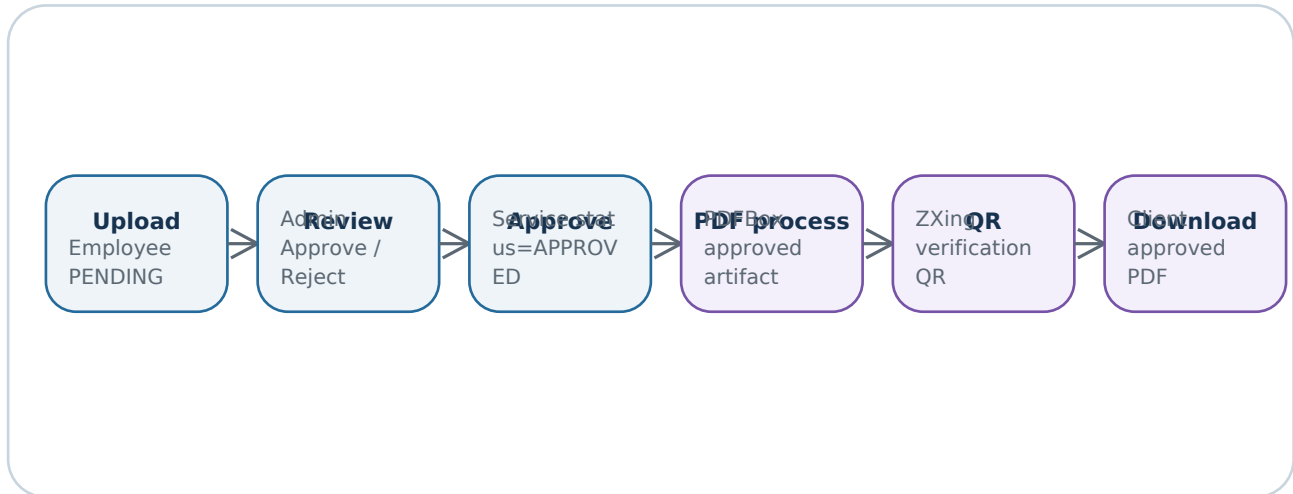
- employee: identity, email, BCrypt password, role and department references.
- user_role: role definitions such as ADMIN and EMPLOYEE.
- department: organizational grouping.
- document: PDF metadata, status, comments, timestamps and approver/uploader references.

Why foreign keys matter

They preserve referential integrity. A document can point to an employee without duplicating employee data, and employee role/department relationships remain explicit.

07 — Document workflow

The business workflow is state-driven. An employee uploads a PDF, the document becomes PENDING, an administrator reviews it, and the decision changes the state. Approval also triggers creation of the approved PDF artifact.



Upload path

React multipart request → document controller → document service → validate employee/file → store source PDF → persist document metadata with PENDING status.

Approval path

Admin request → JWT authorization → service loads document → validates current state → marks approved → PDFBox modifies/creates approved artifact → ZXing produces QR → metadata is updated → approved PDF becomes downloadable.

State is important

A workflow should reject invalid transitions. For example, an already rejected or approved document should not be silently approved again without an explicit business rule.

08 — PDFBox + ZXing

PDFBox is the document-processing component. ZXing is the QR generation component. They are infrastructure concerns used by the service layer rather than being mixed into controllers.

Approved PDF processing

- Load the source PDF with PDFBox.
- Open the final page or appropriate target area.
- Generate a QR image using ZXing.
- Place the QR without creating an unnecessary extra page.
- Save the approved artifact under the project's approved-file naming convention.
- Return the artifact through the download endpoint.

QR purpose

The QR acts as a machine-readable verification handle. The application can encode a verification payload such as document identity/status information. The important architectural idea is that QR generation is server-side and tied to the approval event.

Implementation note

The final design should place the QR on the last existing page rather than creating a new blank/A4 page. Use PDFBox page-content append mode carefully and avoid duplicating an existing signature or visual separator.

09 — Security design

Security is distributed across several controls. BCrypt protects stored passwords. JWT protects stateless API authentication. Spring Security applies the filter chain. CORS controls which browser origins can call the API. Authorization determines what authenticated roles can do.

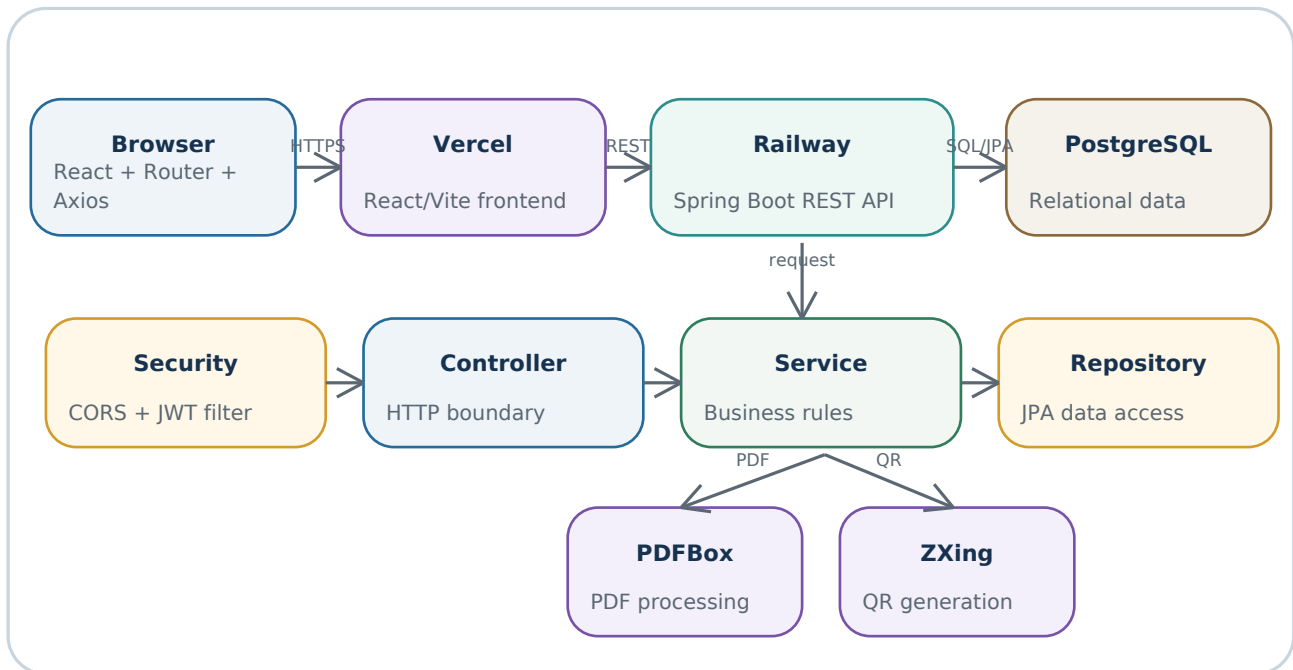
Control	Purpose	Project-level reasoning
BCrypt	Password storage	Store a one-way salted hash, never plaintext passwords.
JWT	Authentication	Carry signed identity/claims between client and server.
Security filter	Request authentication	Validate Bearer tokens before protected endpoints.
CORS	Browser origin control	Allow the deployed frontend origin to call the API.
Authorization	Role/action control	Prevent an authenticated employee from performing admin-only actions.
HTTPS	Transport security	Protect credentials/tokens in transit in production.

Frontend vs backend security

A token check in localStorage can hide a page. It cannot protect an endpoint. The backend must independently authenticate and authorize every protected request.

10 — Docker Compose local architecture

Local development uses three cooperating services: the React frontend, Spring Boot backend and PostgreSQL database. Docker Compose provides a repeatable environment where the services can be started together.



Local ports used during development

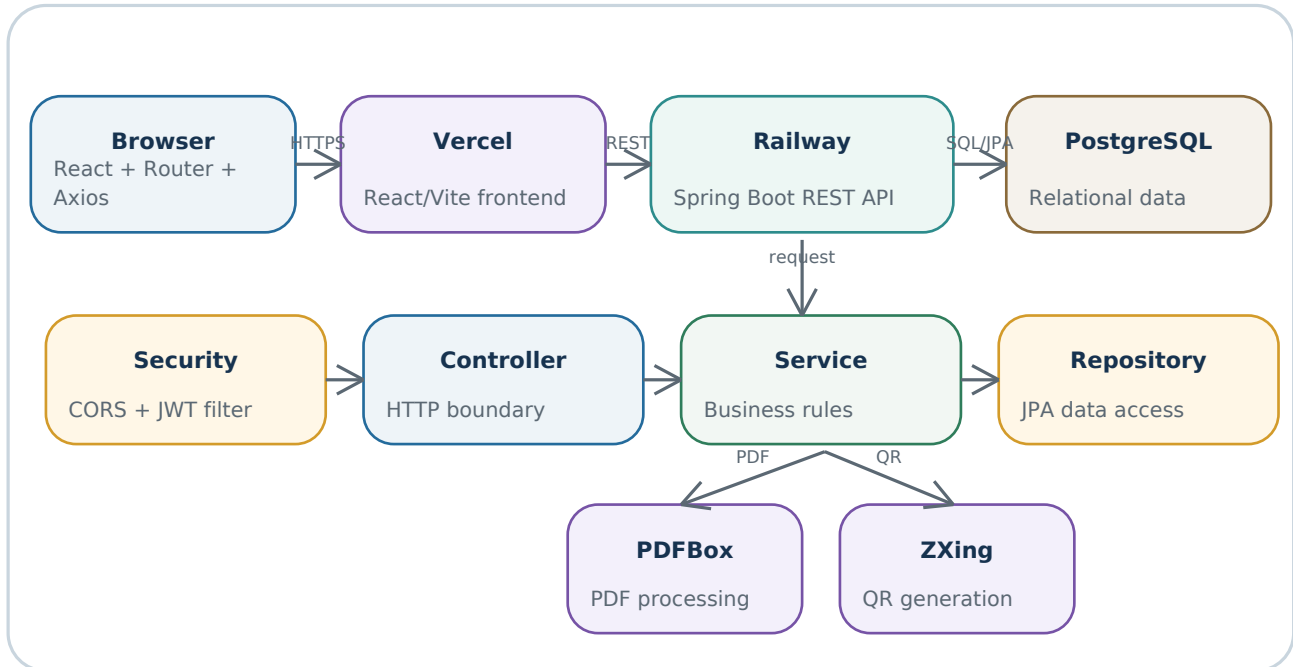
- Frontend: host port 5173 mapped to the container web server.
- Backend: host port 8080 mapped to Spring Boot.
- PostgreSQL: host port 5433 mapped to PostgreSQL port 5432 inside the container.

Environment variables

The application uses environment variables for database connection settings, JWT secret and frontend origin. Local defaults can point to localhost while deployment injects production values.

11 — Production deployment

The deployed architecture separates static/client hosting from the backend runtime and relational database. The React/Vite production build is hosted on Vercel. Spring Boot and PostgreSQL run on Railway.



Production request path

Browser → Vercel frontend → HTTPS API call → Railway Spring Boot → Railway PostgreSQL / PDF infrastructure → HTTPS response

Important deployment configuration

- VITE_API_URL must point to the Railway backend URL at frontend build time.
- FRONTEND_URL must match the Vercel origin used by the browser for CORS.
- SPRING_DATASOURCE_URL, username and password must be supplied by Railway.
- JWT_SECRET must be non-empty and strong enough for the signing algorithm.
- Database schema initialization must be compatible with the deployment environment.

File storage caveat

The backend writes PDFs to a local uploads directory. Cloud filesystem persistence can be ephemeral, so production-grade systems should use durable storage/object storage or a persistent volume.

12 — Debugging playbook

Production debugging is a sequence of evidence collection, not guesswork. Start with the browser Network tab, then correlate the request with backend logs and database state.

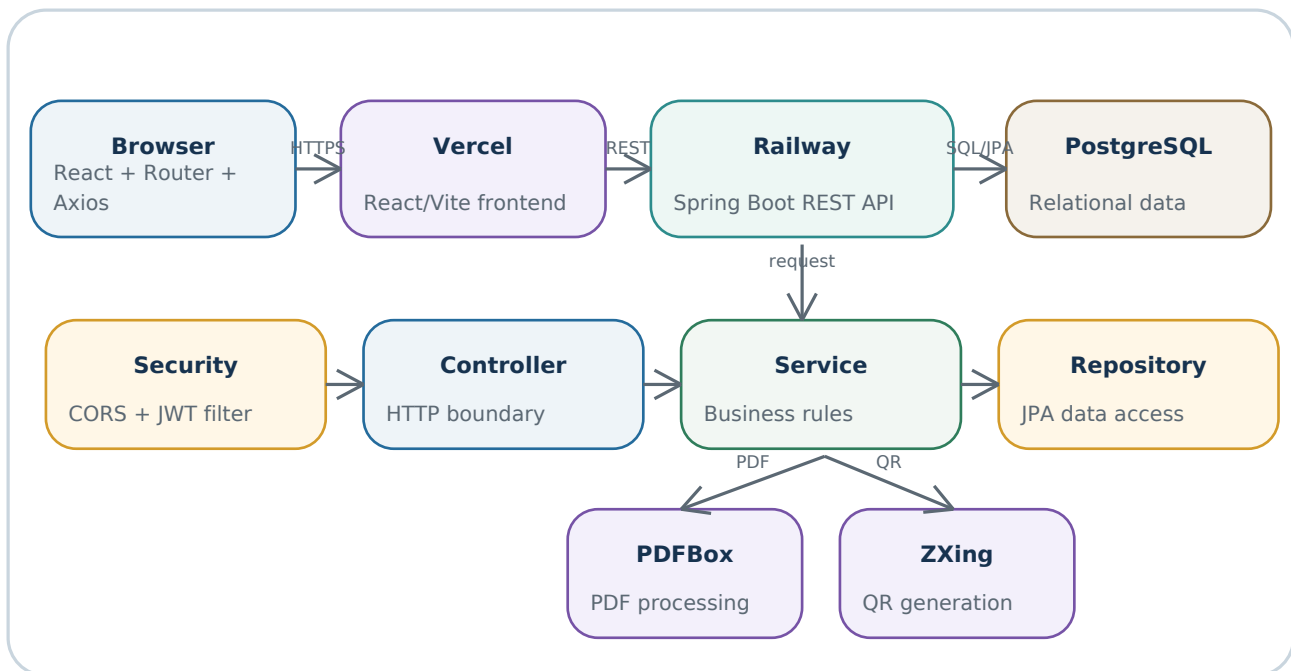
Symptom	Evidence	Likely cause / lesson
Login returns 400	Railway SQL log	Missing employee table/schema not initialized.
JWT WeakKeyException	Startup log	JWT_SECRET is empty or too weak.
Frontend refresh gives 404	Vercel response	SPA fallback/rewrite missing.
CORS error in browser	Network/console	FRONTEND_URL does not match actual origin.
DB connection failure	Spring startup	Datasource URL/credentials/service reference wrong.
Upload succeeds but file later disappears	Filesystem behavior	Ephemeral runtime storage.

Recommended debugging order

1. Reproduce → 2. inspect Network request → 3. inspect backend logs → 4. identify layer → 5. make smallest fix → 6. retest end-to-end

13 — Interview-ready architecture explanation

A strong explanation should be visual and layered. Draw the overall architecture first, then zoom into one request.



60-second version

"I built a full-stack document approval and QR verification system. The React/Vite frontend communicates with a Spring Boot REST backend over HTTPS. Spring Security authenticates requests using JWT and BCrypt, controllers handle HTTP concerns, services implement the approval workflow, and Spring Data JPA persists employee, role, department and document data in PostgreSQL. When an administrator approves a PDF, the backend uses PDFBox for PDF processing and ZXing for QR generation. Locally I use Docker Compose, while production separates the Vercel frontend from the Railway backend and database."

What to memorize

Do not memorize every class. Memorize the request flow, data flow, security flow, approval state flow, and deployment flow.

Final mental map

User → React → HTTPS → Spring Security → Controller → Service → Repository → PostgreSQL

Approval → PDFBox → ZXing → approved PDF → verification

Local → Docker Compose Production → Vercel + Railway